6

其它基础知识

本部分介绍除计算复杂性理论、初等数论、抽象代数和密码学以外的一些基础知识，涉及的领域包括但不限于计算机科学与编程语言学等。

6.1 汉明距离与汉明重量

汉明距离（Hamming distance）和汉明重量（Hamming weight）是计算机科学和通信理论中经常用到的两个重要概念，它们都与字符串的特定属性有关：汉明距离衡量的是两个对象之间的差异；汉明重量衡量的是单个对象自身的特性。在实际应用中，为了提高效率，通常会使用位操作（如 XOR 和位计数）来快速计算汉明重量和汉明距离。

6.1.1 汉明距离

汉明距离是指两个等长的字符串之间对应位置上不同字符的个数。换句话说，它是将一个字符串转换成另一个字符串所需的最小替换次数（只考虑替换，不考虑插入或删除）。汉明距离是衡量两个字符串相似度的一种简单而有效的方法，最常用且高效的方法是借助异或运算，其核心步骤是：先将两个数进行异或操作，然后统计结果中“1”的个数。例如，二进制字符串“1011001”和“1001011”之间的汉明距离是 2，因为有两个位置的位不同（第 3 位和第 6 位）。

6.1.2 汉明重量

汉明重量，也称为汉明重量级或位权重，是指一个字符串中非零符号的个数（对于二进制串，就是“1”的个数），这里的“重量（weight）”是“权重”的意思。汉明重量这一概念在多种应用中都很重要，比如错误检测和纠正、数据压缩以及密码学等。例如，二进制字符串“1011001”的汉明重量是 4，因为它包含 4 个 1。对于一个非全 0 的二进制字符串，其汉明重量等于该字符串与一个等长的全 0 字符串之间的汉明距离。

6.1.3 应用

汉明距离和汉明重量在多个领域都有应用，比如：

- 错误检测和纠正：在通信和数据存储中，汉明距离常用于检测并纠正数据传输或存储过程中引入的错误。通过比较原始数据的汉明重量或计算接收数据与预期数据之间的汉明距离，可以识别并纠正错误。
- 密码学与安全：在密码学中，汉明重量和汉明距离被用于某些加密算法的设计和分析中，以增强安全性或提高算法效率。比如，汉明重量在侧信道攻击分析中很重要，因为电流消耗等物理特征可能与处理数据中的“1”的个数（即汉明重量）相关。汉明距离也可用于衡量密钥或随机数的差异性。
- 数据压缩：汉明重量可用于评估数据压缩算法的效率，因为某些压缩技术试图减少数据中 1 的数量（即降低汉明重量）。
- 图像处理与特征匹配：比如，比较两幅二值化图像（如指纹图像）的相似度时，可以将其视为二维二进制串，可计算汉明距离作为差异指标。

6.2 等重码

本书中使用“等重”（constant-weight）这一说法，一方面是希望与“汉明重量”这一名称中的说法保持一致，另一方面是为了着重强调这类编码具有“汉明重量相等”的特点。

等重码（constant-weight code）也称为定权码、定权重编码，或者“n 中取 m 码”（m-of-n code），是一种所有码字（codeword，这里，一个等重码就是一个码字）具有相同汉明重量的错误检测码。二进制等重码（binary constant-weight code）的所有码字都是二进制字符串，比如独热码（one-hot code）和平衡码（balanced code）等。独热码是一种将分类变量（或称为离散特征、无序特征）转换为二进制向量的方法，常用于数据预处理，尤其在机器学习和数据挖掘领域中；而平衡码是一种特殊的编码方式，其码字中比特“1”的个数和比特“0”的个数是相同的。使用平衡码可以使得传输线路上的信号更

易传送，因为它确保了信号中的正负电平（或称为“1”和“0”）在数量上达到平衡，有助于减少信号在传输过程中的失真和干扰，因此在通信技术、网络技术等领域具有广泛的应用。

对于长度（length，指码字的位数）为 m、汉明重量为 k 的二进制等重码集合 $CW(m,k)$，其大小（size，这里指不同的码字的数量）为 C_m^k（组合数）。若固定汉明重量 k，为了得到大小至少为 n 的二进制等重码集合，必须选择码字的长度 m，使得 $C_m^k \geq n$。

可以使用一些算法将其它定义域的元素映射为等重码，以便于与当前定义域中的元素进行比较。一种等重码映射算法如算法6.1所示。

算法 6.1 等重码映射算法示例

输入： kw, m, k

$h \leftarrow H(kw) \pmod{C_m^k}$ # $h \in [n]$，这里取模以保证范围合理
$a = h$
$b = k$
$CW_{kw} = \{0\}^m$ # 将 CW_{kw} 初始化为长度为 m 的全 0 二进制串
for $m' = m-1, m-2, \cdots, 1, 0$ **do**
    if $a \geq C_{m'}^b$ **then**
        $CW_{kw}[m'] = 1$ # 将 CW_{kw} 的第 m' 位的值置为 1
        $a = a - C_{m'}^b$
        $b = b - 1$
    end if
    if $b = 0$ **then**
        break
    end if
end for

输出： $CW_{kw} \in CW(m,k)$

6.3 相等运算符

检查两个值是否相等是加密数据上的许多任务中不可或缺的一步。相等运算符（Equality Operator）的定义 [1] 如下：过程 f 是定义域 D 上的相等运算符，如果 $\forall x, y \in D$，

有

$$f(x,y)=\begin{cases}1 & \text{if } x=y\\ 0 & \text{o.w.}\end{cases} \tag{6.1}$$

其中，$o.w.$ 是单词 otherwise 的缩写。如果该式以一定的概率 $1-\epsilon$ 成立，则称过程 f 是 ϵ-概率相等运算符。

下面介绍几种常用的相等运算符。

6.3.1 算术 Folklore 相等运算符

算术 Folklore 相等运算符（Arithmetic Folklore Equality Operator）是首个用于比较二进制格式的两个数字是否相等的运算符。对于定义域 $D=\{0,1\}^l$，若用 x 和 y 表示该定义域上的两个二进制数字（显然，这两个数字的位长是相等的），则算术 Folklore 相等运算符的定义为

$$f_{\mathrm{AF}}(x,y)=\prod_{i=0}^{l-1}\left(1-(x[i]-y[i])^2\right). \tag{6.2}$$

该运算符在任意域（比如 $\mathbb{Z}_p$）上都是正确的。实现该运算符的电路的乘法深度为 $\log_2(l)+1$，其中 l 是操作数（operands）的比特长度。该运算符对于两个输入操作数具有不经意性（oblivious），这在某些应用（比如，比较两个加密的或秘密分享的数字）中是至关重要的。当其中一个操作数是已知的时，算术 Folklore 相等运算符可被修正，使之具有更小的乘法深度，修正后的运算符称为朴素 Folklore 相等运算符（Plain Folklore Equality Operator）；当操作是在二进制域上时，算术 Folklore 相等运算符也可被修正，使之具有更小的乘法深度，修正后的运算符称为二进制 Folklore 相等运算符（Binary Folklore Equality Operator）。

6.3.2 朴素 Folklore 相等运算符

朴素 Folklore 相等运算符如何表示依赖于公开的操作数。对于定义域 $D=\{0,1\}^l$，仍用 x 和 y 表示该定义域上的两个二进制数字，且 y 是公开的操作数，则朴素 Folklore 相等运算符可定义为

$$f_{\mathrm{PF}}(x,y)=\prod_{y[i]=0}(1-x[i])\prod_{y[i]=1}x[i]. \tag{6.3}$$

实现该运算符的电路的乘法深度为 $\log_2(l)$。

6.3.3 二进制 Folklore 相等运算符

对于定义域 $D = \{0,1\}^l$，仍用 x 和 y 表示该定义域上的两个二进制数字，则二进制 Folklore 相等运算符可定义为

$$f_{\mathrm{BF}}(x,y) = \prod_{i=0}^{l-1}(1 + x[i] + y[i]) \mod 2. \tag{6.4}$$

实现该运算符的电路的乘法深度同样为 $\log_2(l)$。

6.3.4 二进制 Raffle 相等运算符

二进制 Raffle 相等运算符（Binary-Raffle Equality Operator）是一种概率相等运算符，用于比较二进制域上的二进制格式的元素。对于定义域 $D = \{0,1\}^l$，从中随机抽取 $N(\epsilon)$ 个元素，若用 x 和 y 表示该定义域上的两个二进制数字，并用 r_j，$j \in \{1,2,\cdots,N(\epsilon)\}$ 来表示随机抽取出的元素，则二进制 Raffle 相等运算符可定义为

$$f_{\mathrm{BR}}(x,y) = \prod_{j=1}^{N(\epsilon)}\left(1 + \sum_{r_j[i]=1}(x[i] + y[i])\right) \mod 2. \tag{6.5}$$

若 $N(\epsilon) = \log_2(1/\epsilon)$，该运算符正确输出的概率为 $1-\epsilon$。实现该运算符的电路的乘法深度同样为 $\log_2(N(\epsilon))$，完全取决于该运算符的失败概率，即错误输出的概率。

6.3.5 等重相等运算符

等重相等运算符是基于等重码的相等运算符，其核心思想是：不直接比较两个原始数据本身，而是比较它们对应的等重码码字。等重相等运算符的实现过程包括两个步骤：

- 步骤一（编码）：将需要比较的两个数据 a 和 b，分别通过一个预定的算法，映射（编码）成两个等重码码字 x 和 y。这个编码过程是公开的。
- 步骤二（比较）：比较 x 和 y 是否完全相同。
  - 如果 $a == b$，那么 x 必然等于 y。
  - 如果 $a != b$，那么 x 极大概率不等于 y。由于等重码的特性，即使 a 和 b 不同，它们的编码 x 和 y 也可能偶然相等，但通过精心设计编码方案，可以将这种“碰撞”概率降到极低，满足安全需求。

这种设计带来了一个至关重要的安全属性：恒定时间执行。无论比较的两个数据是相等还是不等，比较操作（即对比两个等重码码字）所花费的时间是恒定的。这从根本上防御了时序攻击。攻击者无法通过测量比较操作所花费的时间长短来推测出数据的任何信息（例如，密码或密钥的哪一部分是正确的）。总之，等重码相等运算符是一种为

了“安全”而设计的比较器。它通过增加固定的、与数据无关的编码和解码开销，牺牲了一点儿绝对效率，换来了对时序攻击的免疫力，是信息安全中“以时间换安全”的典型实践。

这里介绍三种等重码上的相等运算符变体，分别是朴素等重相等运算符、算术等重相等运算符和二进制等重相等运算符。

6.3.5.1 朴素等重相等运算符

对于两个等重码 $x, y \in CW(m, k)$，朴素等重相等运算符（plain constant-weight equality operator）为

$$f_{\mathrm{PCW}}(x, y) = \prod_{y[i]=1} x[i]. \tag{6.6}$$

该运算符对 y 具有不经意性，其结果取决于 x。实现该运算符的电路执行 k 次乘法，乘法深度为 $\lceil \log_2(k) \rceil$。

6.3.5.2 算术等重相等运算符

对于两个等重码 $x, y \in CW(m, k)$，描述算术等重相等运算符（Arithmetic Constant-weight Equality Operator）的算法如算法6.2所示。

算法 6.2 算术等重相等运算符算法

输入： $x, y \in CW(m, k)$

1: $k' = \sum_{i \in [m]} x[i] \cdot y[i]$

2: $e = \frac{1}{k!} \prod_{i \in [k]} (k' - i)$

输出： $e \in \{0, 1\}$

当且仅当 x 与 y 相等时，该算法的输出为 1，否则，输出为 0。实现该运算符的电路执行 $m + k$ 次乘法，乘法深度为 $1 + \lceil \log_2(k) \rceil$。

6.3.5.3 二进制等重相等运算符

二进制等重相等运算符（binary constant-weight equality operator）是定义在二进制域上的，其对应的算法描述如算法6.3所示。

算法 6.3 二进制等重相等运算符算法

输入： $x, y \in CW(m,k), k = (\overline{b_h \cdots b_1 b_0})_2, h = \lfloor \log k \rfloor$

for $i \in [m]$ **do**
  $z_0[i] = x[i] \cdot y[i]$
end for
$b'_0 = \sum_{j \in [m]} z_0[j] \mod 2$
for $i \in \{1, 2, \cdots, h\}$ **do**
  $s_i[0] = 0$
  for $j \in \{1, \cdots, m-1\}$ **do**
    $s_i[j] = s_i[j-1] + z_{i-1}[j-1] \mod 2$
  end for
  for $j \in [m]$ **do**
    $z_i[j] = z_{i-1}[j] \cdot s_i[j] \mod 2$
  end for
  $b'_i = \sum_{j \in [m]} z_i[j] \mod 2$
end for
$e = \prod_{i=0}^{h} (1 + b_i + b'_i) \mod 2$

输出： $e \in \{0, 1\}$

同样地，当且仅当 x 与 y 相等时，该算法的输出为 1，否则，输出为 0。实现该运算符的电路执行 $m + (m+1)\lfloor \log_2 k \rfloor$ 次乘法，乘法深度为 $1 + \lfloor \log_2 k \rfloor + \lceil \log_2(\lfloor \log_2 k \rfloor + 1) \rceil$。如果 k 是 2 的幂次，电路的乘法深度可减小为 $1 + \lfloor \log_2 k \rfloor$。证明从略。

6.3.5.4 主要应用场景

等重相等运算符主要应用于如下场景：

1. 密码学与安全协议：
   - 密码验证：在验证密码或密钥时，使用基于等重码的比较，可以防止攻击者通过分析服务器响应时间的细微差异来破解密码。
   - 防止侧信道攻击：是构建防侧信道攻击密码实现的核心技术之一。
2. 容错计算与硬件设计：在容易出错的通信信道或硬件中，等重码本身可以检测一定程度的错误。基于它的比较器在比较的同时，也附带了一定的错误检测能力。

参考文献

[1] Akhavan Mahdavi R. Equality operators for constant-weight codewords with applications in (keyword) PIR[D]. University of Waterloo, 2021.